San Jose State University

Department of Computer Engineering

CMP240 Advanced Computer Architecture

Midterm Design Document

Project Title: Real-Time Mono Audio Spectrum Analyzer with HDMI Visualization

Semester: Spring 2026 **Date:** 3/31/2026

Team Members:

- Abirami Adaikappan | SID:019153264
- Amina Akhtar | SID: 019118034
- Hemanth Karnati | SID: 01624302
- Shinka Balasundar | SID: 015444065
- Jordan Nguyen | SID: 015364674

---

1. Introduction and Application Overview

1.1 Project Motivation

An audio spectrum analyzer allows the transformation of time-domain signals into a frequency-domain showcase. This lets users see how the energy signals are distributed across various frequencies. This feature is valuable for viewing digital audio processing, sensing, and communication diagnostics. In this case, it is valuable to see the audio signals distributed for audio processing.

The core part of this system is the Fast Fourier Transform, which takes repeated arithmetic operations (internally uses complex butterfly computations) to make calculations and conversions. CPUs can execute FFT processing, but they are limited in the amount of parallelism, and the sequence will be bottlenecked. This creates latency and non-ideal execution for large sizes.

This project aims to use the FPGA structure to map the FFT computation to the PL of the Zynq SoC. With this, we can leverage the pipelined architecture to make the system have deterministic latency, continuous throughput, and parallel execution. This also allows the PS to be focused on control and visualization, and not the heavy computation.

1.2 System Functionality Overview

The system starts with audio captures from the SSM2603 Audio Codec, which is an external hardware component on the Zybo board that acts as the bridge between the FPGA's digital fabric and the analog systems. The Codec will interface with the audio input via the I2S protocol, designed to help sample the input from the audio source to the SSM2603 Codec. This codec is responsible for the very important process of analog-to-digital translation. This translation allows for easy post processing computations for the FFT algorithm. The output of this codec will be a dual channel output that will be fed into the custom IP described below.

The system functionality transitions from raw capture to processing through a Stereo-to-Mono Mixer module. This custom Verilog logic accepts the 24-bit stereo I2S stream (provided by the Audio Receiver) and performs a hardware-level addition (L + R) with an arithmetic right-shift to normalize the signal for spectral analysis. Because the FFT requires discrete frames of data rather than a continuous feed, the resulting mono samples are stored sequentially into a BRAM-based Sample Buffer. Once a complete frame of 256 samples is accumulated in BRAM, the data is encapsulated in an AXI4-Stream interface and routed via the AXI Interconnect IP to the FFT engine. Simultaneously, the system manages user interaction by mapping the physical states of SW0 and SW1 through the AXI GPIO IP. The Processing System (PS) uses software polling to read these states at the start of every 46 FPS render frame, allowing the user to toggle the audio output (SW0) or apply a digital gain scalar (SW1) before the frequency data is written to the HDMI frame buffer.

Afterwards, in the FFT integration, It will be implemented through Xilinx FFT LogiCORE IP with pipeline-streamlined I/O functionality. This lets the system get new samples every clock cycle as the pipeline is filled. Internally, the IP manages this through complex computations of staging and butterfly computations with multiplication factors across the pipeline stages. The FFT IP takes from the AX4-Stream and creates the appropriate AXI4-Stream output. Each frame is shown with a tlast signal, done on the wrapper logic on top of the IP to be utilized by PS. The output itself will be 256 complex samples in fixed-point format. The outputs will then be sent to the AXI DMA through the AXI Stream. The DMA will then write the data directly to DDR through a burst and send an interrupt to

PS. This was done to ensure that there is no polling excessiveness on the CPU, and the burden can be handled by the hardware (high-throughput).

Upon receiving the interrupt from the DMA, the ARM Cortex-A9 Processing System (PS) takes over to handle the data transformation and user control integration. The CPU retrieves the raw complex data from DDR memory, calculates the signal magnitudes, and aggregates these values into discrete frequency bins suitable for visualization. During this software processing stage, the system also checks the states of the hardware switches via the AXI GPIO, applying any user-requested digital gain and adjusting the layout based on the selected display mode. Once the data is scaled and formatted, the PS writes the final bar height values directly into the shared memory-mapped display buffer, preparing the precise visual data needed for the upcoming video frame.

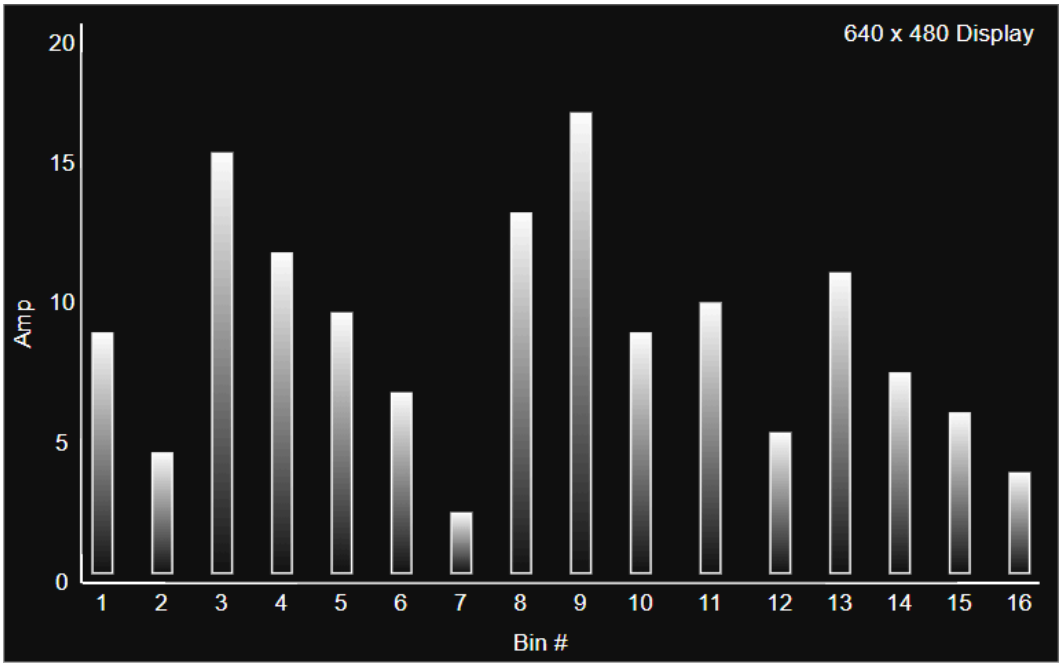
Finally, the HDMI subsystem presents the processed audio spectrum as a real-time bar graph on an external monitor. The Processing System writes the spectrum bar height values into a memory-mapped display buffer in DDR memory, while the Programmable Logic handles the timing-sensitive video path using the Video Timing Controller (VTC), AXI Video DMA, AXI Dynclk, and rgb2dvi IP blocks. The bar graph is formed from grouped FFT frequency bins and displayed as a 640 x 480 image at 60 Hz. This setup allows the spectrum visualization to update smoothly without requiring full-frame software rendering, which will keep the HDMI output stable and efficient to minimize computing on the system.

1.3 Interactive Features

1.3.1 Display Layout and Visual Output:

The HDMI display presents the spectrum analyzer as a centered vertical bar graph as a 640 x 480 cutout on a monitor. The output will use 16 to 32 evenly spaced bars, with each bar representing a grouped range of FFT frequency bins. The bars will rise from a common baseline near the bottom of the screen, making the spectrum easy to read at a glance while leaving enough empty space for easy viewing purposes. The display may also use a simple color gradient, with darker tones near the base and brighter tones near the top of the resolution, to improve readability and make amplitude changes easier to view.

The visualization is designed to be simple and responsive rather than overly decorative. This is because the goal of this project is to demonstrate real-time audio energy, so the display will prioritize the clear bar motion and consistent spacing over complex graphics. This method keeps the HDMI output lightweight and ensures that the visual behavior is easy to spot.



Example Projected Display

1.3.2 Mode Selection Logic:

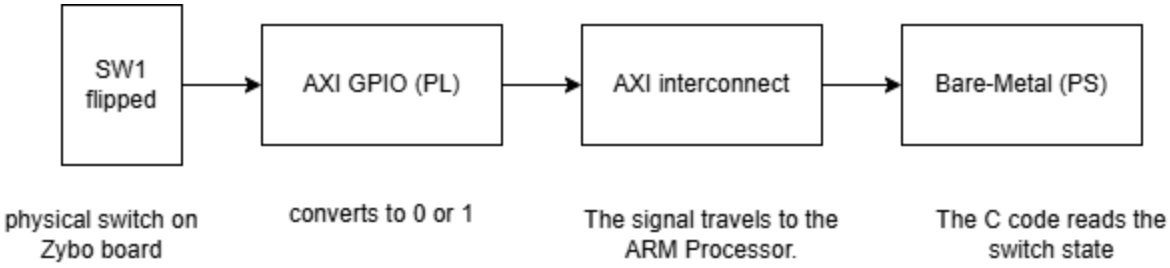
The SW0 (Switch 0) on the Zybo Z7-10 board serves as a binary hardware input. It provides a simple On/Off signal that allows the user to toggle between two different operational states in the system design. The physical sliding switch (SW0) on the Zybo board sends a High (3.3V) signal when flipped up and a Low (0V) signal when flipped down. This voltage is received by the FPGA's Programmable Logic (PL) pins and stored in the AXI GPIO register. In the software, SW0 is read as an on/off flag that can be used to turn on and off the audio codec.

1.3.3 Dynamic Digital Gain Scaling for Visualization:

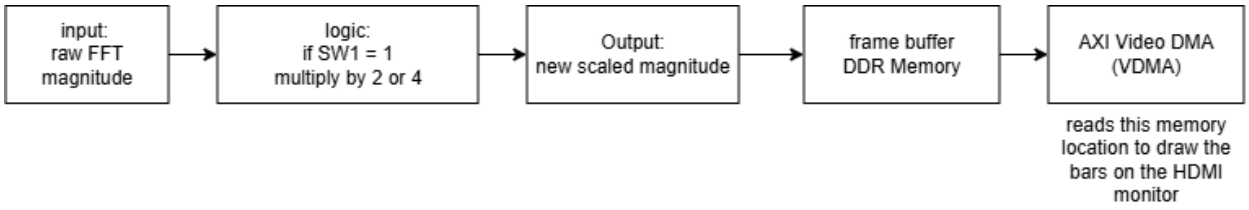
SW1 acts as a mathematical multiplier and is used to control digital gain scaling for the spectrum display. It "boosts" the signal if the input music is too quiet for the bars to jump high enough. It applies a "Scalar" to the raw FFT data.

- If SW1 = 0, the software multiplies the FFT results by 1.0.
- If SW1 = 1, the software multiplies the FFT results by 2.0 or 4.0 (Digital Gain).

The software sends this "scaled" data to the Frame Buffer (DDR Memory). This gain adjustment improves visibility for quieter songs and still allows the HDMI spectrum display to be readable across different types of input levels.



Control Signal Pipeline

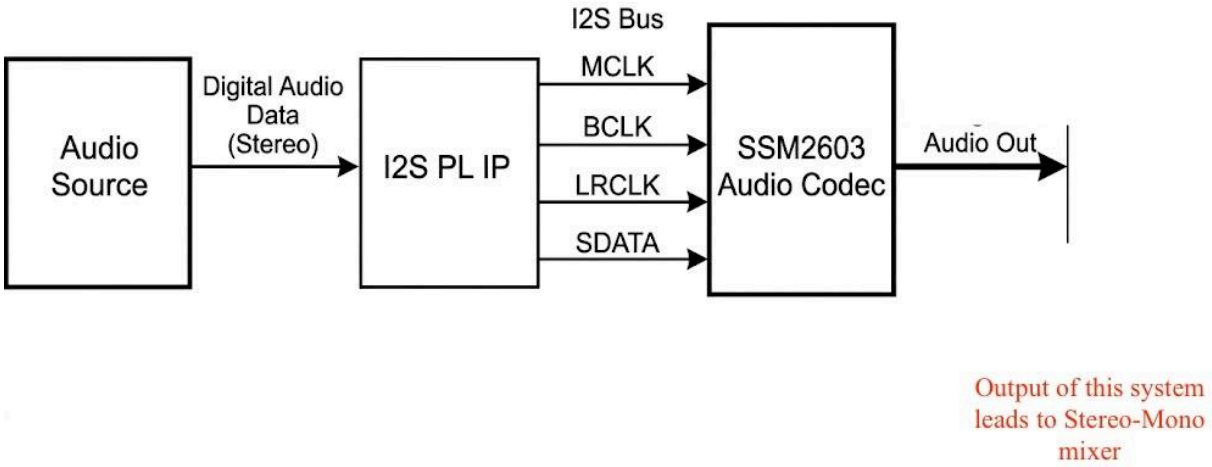


Digital Gain Scaling Data Flow

Ultimately, these switches are interpreted in software to control real-time behaviors. SW0 will handle the start of the flow to enable or disable the audio input, whereas SW1 will adjust the gain. Allowing for real-time manipulation without disrupting the streaming process

1.3.4 Audio Input

The audio input will be fed from a microphone that will be connected to the hardware's 3.5 mm input jack (connector J6). The audio measured through that port will be what gets carried through the rest of the system. The heart of this portion will be the *SSM2603 Audio Codec*, which is responsible for the analog to digital translation. The codec will be controlled by the PS system via I2C, as the following diagram shows.



High-level diagram of audio input structure

2. Requirements and Constraints

2.1 Audio Input Requirements

The input portion to our spectrum analyzer will use a custom Audio IP to receive input from the present input audio jack (J6) that passes through the SSM2603 Audio Codec on the hardware.

The audio IP will interface with the codec via the I2S protocol, capturing audio with a sample rate of 48kHz with a resolution of 24 bits per channel. I2S stands for Inter-IC Sound, a 3 wire protocol designed for carrying sound as sampled binary values.

The previously mentioned I2S protocol will deliver a left and right channel slot with each carrying 24 bits of audio per sample. The I2S protocol is used on the data layer (PL side) by the codec to handle the real-time audio. To control internal components of the codec, the control plane (PS side) uses the Inter-IC Control (I2C) protocol to control the input source, sample word length, and activation.

When audio is transmitted from the Codec, only one of the 2 channels will be carrying the audio, but both left and right channels will present from the stream. Therefore, in order to perform further processing, we must convert the received stereo (dual channels) to

mono (unified singular channel). To achieve this, the output of the audio signal will be forwarded to the ***Stereo to Mono Mixer (7.2.2)***, a custom IP that will perform this conversion. The resulting mono stream will be sent to the frequency-domain transformation and processing.

2.2 Audio Processing and Control Interface Requirements

The Stereo-to-Mono Mixer must function as a Custom AXI4-Stream Wrapper. It shall encapsulate raw 24-bit audio samples into the AXI-Stream protocol to ensure compatibility with the Xilinx FFT IP core.

The interface must utilize TVALID and TREADY handshaking. This constraint is required to prevent “buffer overflow” and ensure zero data loss during high-speed transfers between PL modules.

The hardware architecture must utilize the Xilinx AXI GPIO IP core to bridge the physical SW0 and SW1 signals to the AXI interconnect.

2.3 FFT Processing Requirements

The FFT operates on 256 sample frames (mono audio), which gives 256 complex outputs. If we have a sampling frequency of 48 kHz, that would be: 48000 / 256 = 187.5 Hz/bin. With that, we can also calculate to collect one complete FFT frame is: 256/48000 = 5.33ms. This gives the processing window of the system. The FFT produces 128 unique frequency bins (Nyquist limit), each spaced at 187.5 Hz, covering 0 to 24 kHz. Together, we can get the general update rate of 1 / 5.33 ms, which is 187.5 Hz (It is the theoretical value and does not account for general latencies). But since there are many external factors, we can target a rate of 46 frames per second as our viable product. Since the effective update rate is limited by software processing, HDMI synchronization (60 Hz vsync), and DMA/interrupt overhead, the achievable visualization rate is reduced to ~46 FPS.

When sampling, only the first 128 bins will be utilized since there is symmetry in the FFTs.

The FFT must operate using fixed-point arithmetic. This is a tradeoff required since we want to mitigate overflow risk for less precision. The input samples will be 24-bit signed values, internal scaling will be done with fixed-point arithmetic, and overflow scaling prevention will be done at each stage to try to prevent any overflow case. This gives stability as we preserve the range required for visualization.

2.3.1 Display Requirements and Constraints

The expected output of the system is displayed on an HDMI-connected monitor at 640 x 480 resolution with a refresh rate of 60 Hz. The visualization is implemented as a bar graph using around 16-32 frequency bins. Using this amount will provide us with a compact representation of the audio spectrum while also keeping the display nice and clear for interpretation. The HDMI path has to remain compatible with the selected video timing configuration because the video subsystem depends on a few things: a valid pixel clock, synchronization signals, and a stable stream rate to generate a correct monitor output. In this design, the HDMI chain is built around the Video Timing Controller, AXI Video DMA, AXI Dynclk, rgb2dvi output block, which generates the timing, moves display data, and converts the RGB video stream into HDMI-compatible signaling. In our system, this means the design cannot support arbitrary refresh rate changes or unsupported resolutions without reconfiguring the video timing and clocking logic. If a higher refresh rate or nonstandard mode is requested above the supported configuration, the HDMI pipeline may fail to lock or may exceed the bandwidth that the video blocks can handle. With that said, the display update mechanism should only modify the bar height values in memory rather than redraw the full frame in software. The implementation also has to be synchronized with the AXI-based video interface so that the framebuffer of bar-graph data is delivered consistently without stalling the video or real-time playback.

2.3.2 Display Throughput and Bandwidth

The HDMI display path has to be treated as a continuous real-time video stream rather than a simple output channel because the monitor expects a constant flow of pixel data at the selected resolution and refresh rate. For our chosen resolution and Hz (640 x 480), the subsystem must sustain the required pixel throughput without interruption so that the image remains viewable and the HDMI output stays synchronized. In our system, the bandwidth requirement is kept manageable by updating only the bar graph values that represent the spectrum, instead of rewriting an entire frame from software every cycle. This helps reduce the memory traffic and makes the visualization more efficient, because the display data being modified is small compared with the size of a full video frame. Even though the bar-graph data is very lightweight, the HDMI pipeline still has to maintain valid timing and continuous frame delivery through the video timing and output logic. The result of this is that the main constraint is not the amount of spectrum data being written, but whether the system can handle the HDMI stream without any stalls, timing mismatches, or unsupported video settings.

2.4 Resource and Latency Constraints

2.4.1 Resource Constraints (PL Utilization):

To ensure the design fits comfortably within the Zybo Z7-10's available programmable logic and to reserve sufficient routing resources for the HDMI video subsystem, the architecture aggressively minimizes both Block RAM (BRAM) and DSP slice utilization.

First, by mixing the incoming stereo audio into a single mono channel *before* processing, the design requires only one instantiation of the Xilinx FFT IP core. This cuts the required DSP slices in half compared to a dual-channel stereo processing setup. The FFT IP is also configured to use fixed-point arithmetic, which maps much more efficiently to the FPGA fabric than floating-point logic. Secondly, the hardware sample buffer size is strictly limited to 256 samples. This requires a very small memory footprint (256 samples × 24 bits), utilizing only a fraction of the Zynq's available BRAM tiles and preventing memory bloat.

2.4.2 Latency Constraints (Timing Targets)

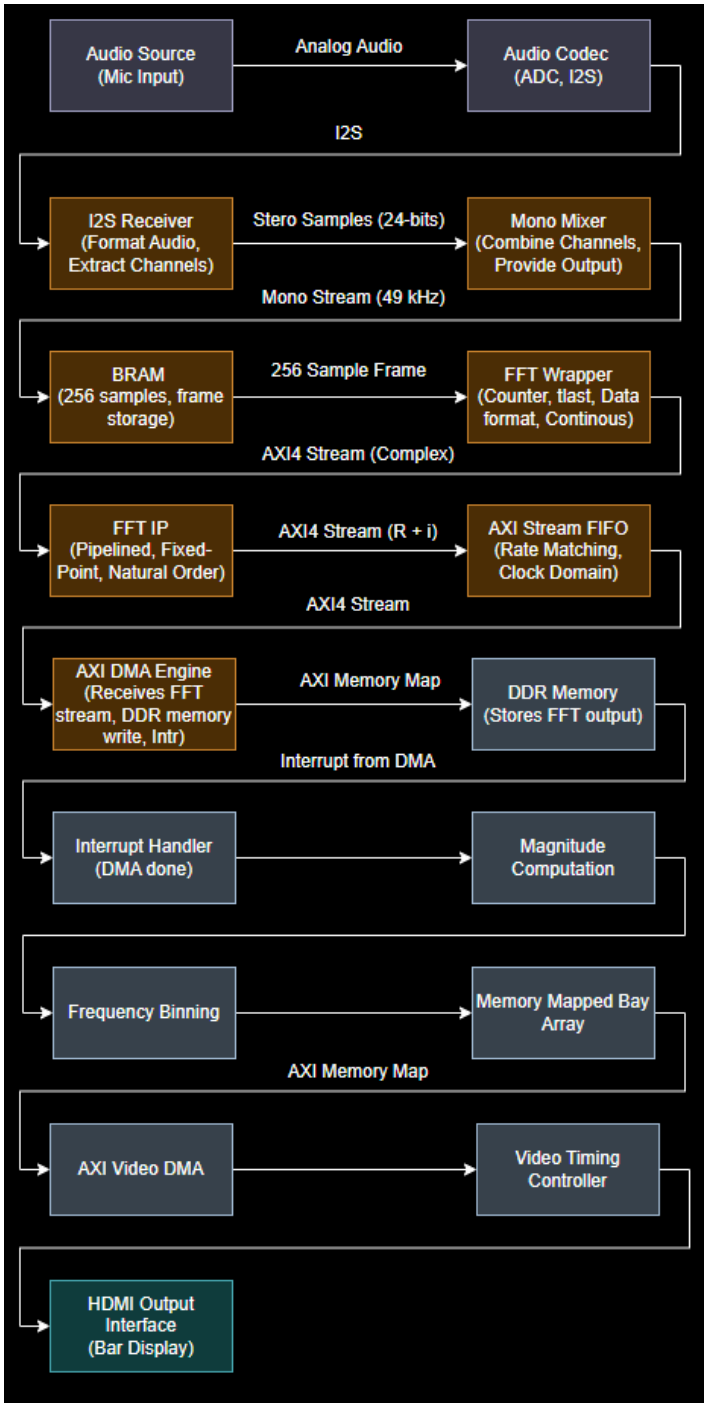
For the visualizer to appear responsive and synchronized with the live audio, the system must maintain a strict total pipeline latency of **< 15 ms** from the moment the audio frame is captured to the moment the display memory is updated. The deterministic timing budget is broken down as follows:

- **Data Acquisition (Wait Time):** Collecting a full 256-sample frame at a 48 kHz sample rate imposes an unavoidable baseline collection time of **≅8 ms** (Calculated as: 256 samples / 48,000 samples per second).
- **Hardware Processing & Data Transfer (PL to PS):** Because the Xilinx FFT IP utilizes a pipelined streaming architecture and the AXI DMA utilizes high-speed burst transfers directly to DDR memory, the mathematical transformation and memory handoff are completed in **< 1 ms**.
- **Software Processing (PS):** The ARM Cortex-A9 must unpack the complex data, compute the absolute magnitudes utilizing its hardware Floating-Point Unit (FPU), and group the values into the 16–32 display bins. This software execution must strictly complete in **< 8 ms**.

Total Estimated Latency: ~15 ms or less, This ensures that for every vertical sync of the monitor, a fresh set of spectral data has been captured, processed by the FFT, and rendered by the CPU, resulting in a zero-stutter, real-time visual experience.

3. System Architecture

3.1 High-Level Data Flow



Full System Diagram Flow

The high-level flow starts with the audio samples, which arrive at approximately 48 kHz. Then the samples are buffered into 256 sample frames (with mono conversion). Frames are then streamed into the FFT IP, which produces complex frequency-domain data. The data is then streamed to the AXI4-Stream. The FIFO absorbs this correctly, so any mismatch is accounted for. Then the DMA transfers the data to the DDR memory while generating an interrupt for PS. The PS then processes the data. Next, the resulting bar heights are written into a memory-mapped buffer in DDR memory, which is used by the HDMI render pipeline to generate the visual spectrum display.

3.2 Hardware/Software Partitioning

Audio:

The Audio implementation is mostly done in PL. This is because the necessary components needed for this implementation have to be done in hardware, as the PS system does not have native support. Another reason is that the high sampling rate and precise timing required for audio processing can only be achieved in PL. The sample buffer is specifically designed in the PL system to avoid jitter, resulting in heavy audio distortion and incorrect sampling. The final reason why this is done in the PL section is to offload the time consuming repetitive tasks from the PS system to defined hardware, allowing the PS system to focus on other critical tasks. The only portion of the Audio implementation that may need to be done in the PS section is controlling and activating the SSM2603 Audio Codec. This is because the Zync hardware system uses the PS side to control the code with the I2C protocol.

Partitioning:

The mono mixer is implemented in the PL to avoid taxing the CPU with 96,000 samples (48 kHz * 2 channels) per second, ensuring a synchronized AXI4-Stream is always ready for the FFT core. AXI GPIO resides in the PL to capture physical switch states; the decision-making logic is handled in PS. This allows software flexibility, enabling instant adjustments to gain multipliers in C code without synthesizing the FPGA bitstream.

FFT:

The FFT implementation is done in PL because of the way it computes. There are a lot of repeated calculations that need to be done and are easily parallelizable. The pipelined hardware allows for multiple stages of this FFT computation to occur, which gives a strong

throughput of 1 sample per clock cycle. The magnitude computation, which has square root operations, benefits from being in PS since more software flexibility is required. If this were to be implemented on the wrapper with FFT, it would cause more hardware complexity. This partitioning is also solidified with the AXI DMA and Interrupts, because it eliminates the CPU polling loops, which give it better performance and data transfer while keeping items scalable for different FFT sizes (if it increases).

Processing System (PS):

The Processing System serves as the mathematical engine and user interface controller for the spectrum analyzer. While the FPGA fabric handles the high-speed data streaming, the PS is used for frame-based tasks where software flexibility is more efficient than building custom hardware circuits. The magnitude computation is partitioned to the PS to take advantage of the ARM Cortex-A9 hardware Floating Point Unit. Calculating the square root of the sum of squares for 128 frequency points in C code is much simpler than implementing equivalent logic in Verilog. This choice allows for quick adjustments to the gain multipliers and frequency weighting without needing to re-synthesize the entire FPGA bitstream. Additionally, the PS manages the AXI DMA orchestration and frequency binning. It reads the raw FFT results from the DDR memory and groups them into the 16 or 32 bars required for the HDMI display. This partitioning ensures that the intelligence of the visualizer, such as responding to the physical switches for gain control, is handled in a flexible software environment while the PL remains dedicated to timing-critical tasks like audio capture and HDMI signal generation.

HDMI:

For the HDMI-related rendering work, the display hardware is implemented primarily in the programmable logic rather than the processing system. The video output chain uses PL-based IP blocks such as the Video Timing Controller, AXI Video DMA, AXI Dynclk, and the rgb2dvi converter to generate the correct timing signals and drive the HDMI monitor. These blocks are responsible for the real-time behavior of the display, including the pixel timing, synchronization, and conversion from the internal RGB video format to HDMI-compatible signaling. The processing system does not generate the HDMI signal directly. It instead updates the spectrum data, applies the gain settings, and writes the bar heights or framebuffer values into memory that the video pipeline reads after. This partitioning is very important because the HDMI output is timing sensitive and has to run continuously without software delays. Since we keep the video timing and signal generation in PL, the design of the system reduces CPU overhead, improves stability, and makes the display easier to integrate with the rest of the real-time audio system.

4. Hardware Design (Programmable Logic - PL)

4.1 Custom Verilog Modules

I2S Receiver:

This I2S Receiver is important because the PS system does not have a native I2S controller, meaning that this must be custom implemented in hardware. The core portions of this module will include the bit synchronization, word alignment, sample capture & validation, and normalization. The bit synchronization is important because the logic must sample the serial data line on the rising edge of the bit clock, while a shift register is used to accumulate the bits into a parallel word. Word alignment is necessary because of the I2S nature of dual parallel channels. The logic must account for *Left-Right Clock* to determine which channel is receiving audio while also recognizing I2S delays. Sample capture and validation is necessary to ensure proper sample captures. This is the core logic that samples audio input and latches it onto a holding register and uses a flag system to ensure that the sampled data is read when all 24 bits have been sampled. Normalization is also important because there needs to be proper formatting to sign-extend the 24 bit audio sample to a 32 bit number. This is because the AXI stream that transfers data across the system is a 32 bit stream, hence ensuring the validity of the sampled data across conversion is of the utmost importance.

Stereo-to-Mono Mixer:

This IP will perform a 25-bit hardware addition (L + R) of the dual 24-bit I2S channels, followed by a 1-bit arithmetic right shift. This process downmixes stereo into a single mono stream while preventing signal overflow and maintaining 48 kHz throughput. By converting to mono in the PL, the module halves the data bandwidth for the FFT computation and ensures resource efficiency. The resulting 24-bit mono samples are then sign-extended to 32 bits and packaged into an AXI4-Stream.

FFT IP Wrapper:

The FFT wrapper is crucial to integrate since we need a layer between the FFT IP and the buffer samples. The IP handles the computation, but it doesn't have the context for the application, which the wrapper is intended for. The core points of the FFT wrapper will include the frame construction. Once the 256 samples have been transmitted, the tlast signal needs to be asserted (to make end of frame), to mark the boundaries. It also needs to handle the AXI4 stream handshake; tvalid needs to be asserted when the data is available, and data is only transmitted

when tready is asserted by the FFT IP. It also needs to handle data formatting since the FFT IP expects complex inputs. For Data Formatting, it converts 24-bit mono samples into a 32-bit complex format by setting the imaginary component to zero. Finally, the wrapper maintains frame-based streaming via internal counters that reset and trigger consecutive frames, targeting a real-time visualization rate of 46 FPS. The output side will create the 16-bit real and 16-bit imaginary components, which are packed together for the 32-bit on AXI4-Stream to AXI DMA.

Sample Buffer (BRAM Interface Logic):

The Sample Buffer acts as the primary synchronization bridge between the real-time audio acquisition and the spectral processing stages by managing data flow through the Block RAM (BRAM) IP. To ensure seamless operation, the module implements a dual-port addressing scheme that allows for simultaneous read and write access.

The Write Logic utilizes a modulo-256 counter that increments with every validated 24-bit mono sample received from the I2S receiver and mixer, sequentially storing the data into Port A of the BRAM. Once the write counter reaches the 255th sample, indicating a full frame is ready, the buffer asserts a "Start" signal to the FFT IP Wrapper. This triggers the Read Logic on Port B, allowing the FFT engine to process a complete, static "snapshot" of the audio window. This hardware-level synchronization is critical; it prevents data collisions and ensures that the FFT engine never operates on a partially updated frame, maintaining the integrity of the real-time frequency visualization. This may also get integrated within the FFT Wrapper.

4.2 Hardware IP Components

Xilinx FFT IP:

This IP will be configured as:

- Transform Length: 256
- Architecture: Pipelined Streaming I/O
- Arithmetic: Fixed Point
- Output: Natural Ordering
- Interface: AXI4-Stream

This allows the FFT to work as a sequence of pipeline stages to do all the internal algorithms. When the pipeline is filled, the FFT will output one item per clock cycle (high throughput).

AXI4-Stream Output & DMA:

The FFT IP will transmit output as tdata = [real(16-bit), imaginary(16-bit)] packed into a 32-bit AXI4-Stream word after fixed-point scaling/truncation.

The AXI Stream FIFO is also utilized to handle the clock and prevent data loss during busy DMA.

The AXI DMA is utilized in stream-to-memory mode:

- It will read the FFT output stream
- Write to the DDR
- Use burst transfer

The interrupt will also be generated from the DMA after transferring a full frame. This will notify the PS, allowing for a full asynchronous model during transfer.

Block Memory Generator (BRAM):

- Purpose: High-speed internal storage for FFT frames.
- Configuration: True Dual-Port, 24-bit width, 256-word depth. It provides a low-latency interface that allows the PL to store samples and the FFT (or PS) to read them simultaneously.

Video Timing Controller (VTC):

This IP generates the horizontal and vertical synchronization timing required for HDMI display mode. It is responsible for producing a valid video timing structure so that the monitor receives a stable 640 x 480 on 60Hz signal. The VTC makes sure the video pipeline stays aligned with the selected resolution and refresh configuration.

AXI Dynclk:

This IP will be used to generate the clock signals required by the HDMI video path. It produces the pixel clock and related timing clocks needed to match the selected video mode. Since the HDMI output depends on a precise and constant clock frequency, this block is needed to keep the VTC and video stream synchronized. If the clock isn't configured correctly, the display might not be able to lock or may show an unexpected image on the screen.

Rgb2dvi:

This IP converts the internal 24-bit RGB video data as well as the pixel clock and synchronization signals, into HDMI/DVI compatible TMDS signaling. It acts as the last

output encoder in the display chain and is needed for sending the video stream from the FPGA to the HDMI monitor. Without this IP, the raw RGB video signal is unable to transmit over the HDMI connector to the display.

Clocking Wizard:

This IP will be used to set the clock for sampling the audio by transforming the board's system clock to a master clock frequency. This clock output will drive the Audio Codec chip on the board and drive the I2S IP logic.

I2S Receiver IP:

This IP will be used to sample the audio sent from the audio codec as a serial bitstream and will be responsible for tracking the BCLK (bit clock) and LRCLK (Left/Right clock). The IP will use a shift register to shift a bit every rising edge of the BCLK. Once all 24 bits have been shifted into the register, a VALID signal will be set, and the parallel logic will be shifted to the Mono mixer.

FIFO Generator (Clock Domain Crossing):

This IP will be responsible for ensuring that the main system does not slow down because of the audio sampling. This is because the clocking used for the audio system will be slower than the main system clock, and if not separated properly, could cause data corruption. To prevent this, this IP will read in the data with the slow audio clock and output the data with the fast system clock.

5. Software Design (Processing System - PS)

5.1 Magnitude Computation and Frequency Binning

The Processing System (PS) acts as the data transformation engine of the spectrum analyzer. Its primary software responsibility is taking the raw, complex mathematical outputs from the FPGA's Fast Fourier Transform (FFT) block and converting them into scaled, visual data suitable for an HDMI display. This process occurs in two primary stages: magnitude computation and frequency binning.

5.1.1 Magnitude Computation Logic

The Xilinx FFT IP core processes a 256-sample frame and outputs 128 usable frequency bins (restricted by the Nyquist limit, representing frequencies up to 24 kHz). To optimize AXI bus

bandwidth, the Programmable Logic (PL) packs the output of each frequency bin into a single 32-bit register.

The bare-metal C application reads this mapped memory and must unpack the data to extract the 16-bit real (Re) and 16-bit imaginary (Im) components. This is achieved using bitwise shifting and masking.

Because the FFT outputs complex numbers that represent both amplitude and phase, the PS must calculate the absolute magnitude (the "loudness") of each frequency bin. The phase information is discarded. The magnitude is calculated using the Pythagorean theorem:

$$|X[k]| = \sqrt{Re[k]^2 + Im[k]^2}$$

For the implementation on the Zynq-7000, the ARM Cortex-A9 includes a hardware Floating-Point Unit (FPU). Because of this, calculating the precise square root using standard C libraries (sqrtf()) 128 times per frame easily executes within the required < 15 ms latency window, avoiding the need for complex, CPU-intensive approximation algorithms.

5.1.2 Frequency Binning Algorithm

Once the 128 linear magnitudes are computed, they must be spatially compressed to fit the constraints of the 16- to 32-bar HDMI display. Drawing 128 distinct vertical bars would result in a cluttered and visually noisy interface.

The software utilizes a grouping algorithm to condense these magnitudes into a smaller array of display bins.

The most straightforward approach is linear binning, where the 128 frequency bins are divided evenly among the display bars. For a 16-bar display, the grouping factor is 128÷16=8 bins per bar.

5.1.3 Software Implementation and Libraries

The software application is developed in C using the Xilinx Standalone (Baremetal) Library. This library provides the low-level hardware abstraction layer (HAL) and drivers necessary to initialize the AXI DMA, configure the AXI GPIO for switch inputs, and manage the Video Timing Controller. For the magnitude calculations, the standard Math.h library is used to leverage the hardware FPU for square root functions. While these libraries handle the hardware communication, the Frequency Binning algorithm, Gain Control logic, and HDMI Bar-Update routines are original logic written from scratch.

5.2 HDMI Rendering and Memory Mapping

The software updates the HDMI display by writing bar height values into the memory-mapped framebuffer or bar array that is stored in DDR memory. After the FFT magnitudes are computed and put into display bins, each bin is converted into a bar height value that represents the current energy level of that frequency range. The bar height values are written into a fixed memory region that will act as the display buffer for the visualizer. Because the bar data changes from frame to frame, the software does not need to redraw the entire screen or generate pixel-level graphics manually. Instead, it uses the method of updating a compact set of values that the video pipeline can use to produce the image on the monitor.

The HDMI output itself is handled by the video subsystem in the PL, which includes the Video Timing Controller and AXI Video DMA. The VTC generates the horizontal and vertical timing signals needed to maintain a valid HDMI display mode, while the AXI Video DMA transfers the framebuffer or bar-array data from memory onto the video stream. This allows the display hardware to continuously output a valid image without requiring the CPU to manage every pixel. The processing system only modifies the bar data in memory and lets the hardware video path take care of the timing and transmission of the visual output on the monitor.

5.3 AXI GPIO Polling Scheme:

The system utilizes software polling within the main execution loop to read the physical state of the Zybo switches via the AXI GPIO register. At the start of every frame (~46 FPS), the ARM processor reads the 2-bit switch value to determine the current system configuration.

Polling is preferred in this part of the design since the switches are non-critical, low-speed inputs. Since the CPU is already running a high-speed loop to update the HDMI visualization, it can check the GPIO address with zero overhead, avoiding the complex "Generic Interrupt Controller" (GIC) setup required for an interrupt-driven design.

6. Data Communication and Interfaces

6.1 Communication Protocols

The HDMI output path uses the video subsystem instead of the direct pixel-by-pixel software render method. Using this method, the processing system writes the spectrum

bar values into a memory-mapped display buffer, and the programmable logic handles the timing-sensitive work required to send the video stream to the monitor. The video subsystem uses dedicated hardware blocks to maintain the correct display timing, generate valid synchronization signals, and convert the internal RGB video data into an HDMI-compatible output stream. This separation is crucial because the monitor requires a continuous video signal, while the processor only needs to update the visual data that changes from frame to frame.

The I2S protocol is used by the Audio IP to bridge the FPGA's digital logic with the analog systems of SSM2603 codec. I2S is a point-to-point protocol designed specifically for the continuous, jitter-free transmission of audio data. It relies on a three-wire primary architecture: the Bit Clock (BCLK), which synchronizes each individual bit of data; the Word Select (LRCLK), which toggles to indicate whether the current data belongs to the left or right audio channel; and the Serial Data (SDATA) line, where the audio samples are transmitted MSB-first.

The I2C protocol is used by the PS system to control the SSM2603 codec. While the PL system handles the actual sampling and high speed stream, the PS system is responsible for the initial handshake between the PS system and the codec as well as the configurations.

AXI Interconnect is used to bridge the AXI GPIO to the processor. This allows the software to poll the physical states of SW0 and SW1 every 21ms via Memory-Mapped I/O. By isolating these control signals from the high-speed audio data path, the system can adjust visualization parameters like gain and audio toggling in real-time without causing latency in the hardware pipeline.

AXI4 Stream is utilized as the high-speed protocol for all the status signals between the various critical IPs.

6.2 Data Rates and Throughput

FFT:

The FFT output has 256 complex samples with each sample having 32-bits in total.

$$Frame\ Size\ =\ 256\ \times\ 4\ bytes\ =\ 1\ KB\ per\ frame$$

At an update date of approximately 46 frames per second, the result would be around 46 KB/s. This is well below the bandwidth limits of the AXI DMA and DDR memory subsystem, ensuring that no data transfer bottleneck occurs.

HDMI:

The HDMI display side is not a big bandwidth bottleneck because the software updates only the bar graph data instead of constantly rewriting the full video frames all at once. At 640 x 480 and 60Hz, the monitor will only refresh the screen at a stable rate while the application only changes the visual bar values in memory as the new FFT frames are being processed. Doing it this way will keep the HDMI side lightweight and allow the system to maintain real-time performance for our use case.

Audio:

$$Data\ Rate\ =\ 48,000\ \times 24\ \times 2\ =\ 2.304\ MBps$$

The formula above calculates the expected data rate for the audio IP by multiplying 3 key factors: sampling rate, sample granularity (bits), and # of channels. The resulting answer provides the necessary minimum speed that the I2S bus needs to handle to properly sample at 48kHz with a 24 bit granularity.

$$Throughput\ =\ 32\ \times 2\ \times 48,000\ =\ 3.072MHz$$

The formula above calculates the expected throughput for the audio IP by multiplying 3 key factors: the # of bits per frame, the # of channels, and the sampling rate. The answer provides the total throughout of the I2S bus.

6.3 Buffer and FIFO Sizing

The Sample Buffer is implemented using Block RAM (BRAM) within the FPGA fabric. Its primary purpose is to act as a data accumulation bridge between the slow, continuous audio input and the fast, block-based FFT processing engine. Because the Xilinx FFT IP cannot process individual audio samples on the fly, the BRAM acts as a waiting room, collecting incoming 24-bit mono audio samples at a rate of 48 kHz.

Once the BRAM accumulates exactly 256 samples (representing one complete FFT frame of ~5.3 ms), it triggers the FFT Wrapper to flush the entire block of data into the FFT IP at high speed via an AXI4-Stream interface. By restricting the buffer to 256 samples, the memory footprint remains minimal (256 samples × 24 bits), utilizing only a fraction of the Zynq's available BRAM tiles and preserving FPGA fabric for routing and other IPs.

7. Development Plan

Phase	Task
Week 1 - 2	Audio Input (I2S + Codec Setup)
Week 2 - 3	Stereo-to-Mono + Buffer (BRAM)
Week 3 - 4	FFT Integration (AXI4-Stream)
Week 4 - 5	DMA + Interrupt Setup
Week 5 - 6	Magnitude + Binning (C Code)
Week 6 - 7	HDMI Pipeline + Framebuffer
Week 7 - 8	System Integration + Debug
Last Week	Testing + Optimization